Tutorial 2

Open `src/tutorial-2/Tutorial2.tsx` to get started.

How to Add Step-by-Step Walkthroughs

Why stepping?

In Tutorial 1, you built an interactive formula where readers could drag variables and watch results update. But for multi-step calculations, seeing just the final answer isn't enough — readers need to understand *how* you get there.

Stepping solves this. Instead of showing everything at once, you guide readers through a calculation one piece at a time. At each step, parts of the formula highlight and labels appear to show intermediate values. Readers control the pace with navigation buttons.

What this tutorial covers

This tutorial walks you through how to add step-by-step walkthroughs to an existing formula. By the end, you'll be able to turn any multi-step calculation into a guided walkthrough.

This tutorial builds on Tutorial 1. You should already know how to create a config with `formulas`, `variables`, and `semantics`.



A note on feedback

As you work through this tutorial, please think aloud — narrate what you're doing, what you expect to happen, and what you notice along the way. There are no wrong

observations. We're interested in hearing your thought process as you learn the library, not just the end result.

Starting point

Open `Tutorial2.tsx`. You'll find a working gravitational force formula already set up — the same one from Tutorial 1:

- **Formula**: $\vec{F} = G\frac{m_1 m_2}{r^2}$
- **Variables**: `G`, `m_1`, `m_2`, `r`, and `\\vec{F}` with names, defaults, and formatting
- **Semantics**: Computes the gravitational force from the input variables

The formula displays and computes correctly, but there's no stepping — readers see everything at once. Your goal is to add step-by-step navigation so readers can follow the calculation one piece at a time.

Step 1: Enable stepping

Tell the library that this formula should support step-by-step navigation. **Add `stepping: true` to your config and add `step` to the semantics function parameters:**

```
const config: Config = {
  formulas: [...],
  variables: {...},
  stepping: true,
  semantics: function ({ vars, step }) {
    // your existing computation goes here
  },
};
```

`stepping: true` turns on step mode for the formula. Adding `step` to the destructured parameters gives you a function you can call to create snapshots at key points in the calculation.

Step 2: Add your first step

Decide where the first natural breakpoint is in your calculation. **After computing the mass product, add a `step()` call:**

```
semantics: function ({ vars, step }) {
  var m1 = vars.m_1;
  var m2 = vars.m_2;
  var product = m1 * m2;
  step({
    labels: {
      m_1: m1,
      m_2: m2,
    },
  });
  // ... rest of the computation
},
```

Each `step()` call captures a snapshot that the reader can navigate to — think of it as a "pause here and explain" moment. The `labels` object controls what is shown at this step. Here, `m_1: m1` and `m_2: m2` are **variable labels** — they display the current values of those variables.

What if `step()` is called inside a loop?

Every `step()` call creates a separate navigable step, regardless of whether it's written out individually or called inside a loop. If `step()` runs three times in a `for` loop, the reader gets three steps to navigate through — one for each iteration. The library doesn't deduplicate or merge calls; it simply collects them in order. This means you can use loops to generate steps dynamically — for example, showing each iteration of an algorithm, or building up a series term by term.

This pairs naturally with **array-valued variables**. A variable's `default` can be an array instead of a single number, which is useful for formulas involving summations or indexed data. You can then loop over the array and call `step()` at each iteration:

```
variables: {
    X: {
        default: [10, 20, 30],
        name: "Data values",
    },
```

```
},
semantics: function ({ vars, step }) {
    var xValues = vars.X;
    for (var i = 0; i < xValues.length; i++) {
        var xi = xValues[i];
        step({
            description: "Process value " + xi,
            labels: { X: xValues },
        });
    }
    // use xi in computation...
}
```

Step 3: Add a description

Give readers text that explains what's happening at a step. **Add a `description` to your second step, after squaring the distance:**

```
  var r = vars.r;
  var squared = r * r;
  step({
    description: "Square the distance",
    labels: {
      r: r,
    },
  });
```

`description` is text shown to the reader for this step. It supports inline LaTeX with `$...$` (e.g. `"Square the distance: $r^2$"`).

How does `description` differ from `labels`?

`description` and `labels` serve different roles and appear in different places:

- `description` is a sentence or phrase that appears *outside* the formula. It explains the purpose of the step — the "why" or "what we're doing." Think of it as a caption or narration.

- `labels` appear *on the formula*, attached to specific variables or expressions. They show concrete values or short annotations directly next to the relevant symbols — the "what the numbers are."

Both can appear in the same step. A step can have a `description` only, `labels` only, or both together. Use `description` when you want to narrate or summarize what's happening ("Square the distance"), and use `labels` when you want to show specific values or highlight parts of the formula (`r: r`, `"r^2": squared`).

Step 4: Highlight parts of the formula with expression labels

So far, labels have used variable keys like `m_1` and `r` to show values on individual variables. But you can also highlight an entire *portion* of the formula by using an **expression scope** — a LaTeX substring that matches part of the formula.

Add an expression label to highlight $m_1 m_2$ in the formula:

```
var product = m1 * m2;
step({
  labels: {
    m_1: m1,
    m_2: m2,
    "m_1 m_2": "Multiply the two masses",
  },
});
```

The key `"m_1 m_2"` is not a variable key — it's a substring of the formula's LaTeX (`"\\vec{F} = G \\frac{m_1 m_2}{r^2}"`). Because it matches part of the formula, it highlights that portion and displays the label text next to it.

Expression scope strings must be exact substrings of the formula's LaTeX. For this formula:

- `"m_1 m_2"` highlights the mass product in the numerator
- `"r^2"` highlights the squared distance in the denominator
- `"\\frac{m_1 m_2}{r^2}"` highlights the entire fraction

Step 5: Format numbers with the `latex()` helper

Raw JavaScript numbers can look awkward in labels — too many decimals, no scientific notation. **Import `latex` from the library and use it to format a computed value in your label:**

```
import { latex } from "math-notation";
```

```
  var product = m1 * m2;
  step({
    labels: {
      m_1: m1,
      m_2: m2,
      "m_1 m_2": "Multiply the two masses = " + latex(produc
t).sigfigs(3),
    },
  });
```

`latex(product).sigfigs(3)` formats the number with 3 significant figures in mathematical notation. You can concatenate it with a descriptive string to create readable labels.

For decimal places instead of significant figures, use `.precision()`:

```
latex(someValue).precision(2)  // e.g. "3.14"
```

Step 6: Build out the remaining steps

Now apply what you've learned to the rest of the calculation. **Add `step()` calls for squaring the distance, dividing, and multiplying by G.** Here is the full semantics function with all four steps:

```
semantics: function ({ vars, step }) {
  var G = vars.G;
  var m1 = vars.m_1;
  var m2 = vars.m_2;
  var r = vars.r;

  // Step 1: Multiply the two masses
```

```
  var product = m1 * m2;
  step({
    labels: {
      m_1: m1,
      m_2: m2,
      "m_1 m_2": "Multiply the two masses = " + latex(produc
t).sigfigs(4),
    },
  });

  // Step 2: Square the distance
  var squared = r * r;
  step({
    description: "Square the distance = " + latex(squared).si
gfigs(4),
    labels: {
      r: r,
    },
  });

  // Step 3: Compute the final force
  var fraction = product / squared;
  var force = G * fraction;
  vars["\\vec{F}"] = force;
  step({
    description: "Multiply by $G$ to get force",
    labels: {
      "\\vec{F}": latex(vars["\\vec{F}"]).precision(2),
        G: vars.G,
      "m_1 m_2": latex(product).sigfigs(4),
      "r^2": latex(squared).sigfigs(3),
    },
  });
},
```

Each step introduces one computation and uses labels to show what's happening — variable labels for individual values, expression labels to highlight and annotate parts of the formula.

Step 7: Add the StepControl component

With steps defined, readers need a way to navigate through them. **Add `StepControl` to your JSX, alongside `Formula`:**

```
import { Formula, Provider, StepControl } from "math-notatio
n";
```

```
<Provider config={config}>
  <Formula id="gravity" style={{ height: "400px", width: "800
px" }} />
  <StepControl />
</Provider>
```

`StepControl` renders navigation buttons (skip to start, previous, next, skip to end) and a progress bar. It must be inside `Provider`.

Reference

The `step()` function

Called inside `semantics` to define computation steps. Each call creates a navigable snapshot.

```
step({
  description: "Text with $LaTeX$ support",
  labels: {
    "varKey": value,
    "expression scope": "Label text",
  },
});
```

Property	Description	Required?
`description`	Step text shown to the user. Supports inline LaTeX with `$...$`.	No
`labels`	Variable keys or expression scopes mapped to display values.	No

Label keys

Key type	What it does	Example
Variable key	Shows the value on that variable's label	`m_1: m1` — displays the value of m_1
Expression scope	Highlights that portion of the formula and shows a label	`"m_1 m_2": "Product"` — highlights $m_1 m_2$

Label values

Value type	Example	Result
Number	`m_1: vars.m_1`	Displays the number
Formatted number	`latex(product).sigfigs(3)`	Displays with specified sig figs
String	`"Multiply the masses = " + ...`	Displays text, supports inline LaTeX
`null`	`"m_1 m_2": null`	Highlights expression, no label text

The `latex()` helper

Method	Description	Example
`.sigfigs(n)`	Format with n significant figures	`latex(value).sigfigs(3)`
`.precision(n)`	Format with n decimal places	`latex(value).precision(2)`

New config options

Option	Description
`stepping`	Set to `true` to enable step-by-step navigation.

New components

Component	Purpose
`StepControl`	Navigation buttons for stepping through (previous, next, skip, etc.)